KOHEM CHEMICALS

Documentación Técnica y Funcional del Frontend

MVP de gestión de materias primas, pedidos y asesoría

Versión: 1.0

Fecha: 20 de octubre de 2026

Alcance: SPA React + TypeScript conectada al backend FastAPI existente

Ubicación: MVP/frontend

Contenido

1. Resumen ejecutivo
2. Contexto y alcance
3. Tecnologías, frameworks y lenguajes
4. Arquitectura del frontend
5. Autenticación y sesión
6. Conexión con el backend
7. Servicios API y payloads
8. Funcionalidades y pantallas
9. Roles y permisos
10. Reglas de negocio reflejadas en la interfaz
11. Manejo de errores y estados de interfaz
12. Seguridad del frontend
13. Pruebas y verificación
14. Ejecución y uso
15. Limitaciones y trabajo futuro
16. Conclusiones
17. Anexos

1. Resumen ejecutivo

Se desarrolló una aplicación web de una sola página (SPA) para Kohem Chemicals que consume el backend FastAPI existente sin modificar su código. El frontend cubre el flujo operativo disponible en la API real: autenticación con JWT, consulta y administración de materias primas, registro y seguimiento de pedidos, asesoría mediante un agente IA simulado y consulta de logs de auditoría.

La solución se construyó con React 19, TypeScript estricto, Vite 8, Tailwind CSS 4, React Router, Axios, React Hook Form y Zod. La lógica de acceso a la API está separada de los componentes visuales mediante una capa de servicios, tipos y esquemas de validación. La interfaz adapta menús, rutas y acciones al rol autenticado (`admin`, `analista`, `cliente`), manteniendo al backend como autoridad final de autorización.

El resultado es un MVP funcional, responsive y verificable:

- Compilación de producción correcta.
- Lint sin advertencias.
- 36 pruebas automatizadas en verde.
- Pruebas de humo contra el backend real con usuarios `admin`, `analista` y `cliente`.
- Documentación de las limitaciones del backend y de los cambios necesarios para producción.

> **Importante:** el prompt inicial mencionaba usuarios, citas, geografía, refresh tokens, rate-limit y borrado lógico. Esos elementos **no existen en el backend real**, por lo que no se inventaron endpoints ni pantallas funcionales para ellos. Se documentaron como trabajo futuro.

2. Contexto y alcance

2.1 Backend real

El backend del repositorio es una API FastAPI con las siguientes características:

- Persistencia con SQLite y SQLAlchemy asíncrono.
- Autenticación con JWT firmado en HS256.
- Contraseñas con bcrypt mediante passlib.
- Roles reales: `admin`, `analista`, `cliente`.
- Prefijo de negocio: `/api/v1`.
- Access token con expiración de 60 minutos.
- Sin refresh token, sin revocación y sin logout del servidor.
- Sin CORS configurado.
- Sin rate-limit ni bloqueo de intentos de login.

2.2 Alcance implementado

Módulo	Estado	Observación
Login y logout local	Implementado	JWT en `sessionStorage`; logout solo del cliente
Registro de clientes	Implementado	Sin selector de rol; el backend asigna `cliente`
Dashboard por rol	Implementado	Métricas calculadas con hasta 100 registros de la API
Materias primas	Implementado	Consulta, creación, edición y eliminación física
Pedidos	Implementado	Creación, historial, detalle, cancelación y cambio de estado
Asesoría IA	Implementado	Agente simulado por palabras clave
Auditoría	Implementado	Solo lectura para `admin` y `analista`
Mi sesión	Implementado	Información del JWT; no existe edición de perfil
Usuarios	No soportado	No existen endpoints de usuarios
Citas	No soportado	No existe recurso `appointments`
Geografía	No soportado	No existen países, departamentos, ciudades ni sectores
Refresh token	No soportado	El backend solo emite access token
Rate-limit / lockout	No soportado	No implementado en el backend
Soft delete	No soportado	`DELETE` de materiales es físico

2.3 Principios de diseño

1. **El backend es la fuente de verdad.** La interfaz no inventa reglas ni datos.
2. **Separación de responsabilidades.** Los componentes visuales no realizan llamadas HTTP directas.
3. **Tipado estricto.** Los tipos reflejan los DTOs reales y los esquemas Zod replican las validaciones.
4. **Defensa en profundidad.** La UI oculta acciones no permitidas, pero el backend valida cada operación.
5. **Mensajes claros.** Los errores técnicos se transforman en mensajes accionables para el usuario.

6. **Seguridad por defecto.** Sin secretos en el código, sin `dangerouslySetInnerHTML` y sin logs de tokens.

3. Tecnologías, frameworks y lenguajes

3.1 Lenguajes

Lenguaje	Uso	Justificación
TypeScript	Código de la SPA	Tipado estático, autocompletado y detección temprana de errores en contratos de API
HTML5	Estructura de la aplicación	Base estándar de la SPA y del documento generado
CSS (Tailwind)	Estilos y diseño responsive	Utilidades atómicas, consistencia visual y rapidez de implementación
JSON	Payloads de API y configuración	Formato nativo del backend FastAPI
PowerShell	Script `dev.ps1`	Automatizar el arranque conjunto de backend y frontend en Windows
Markdown	Documentación	Fuente legible y mantenible para generar el PDF

3.2 Frameworks y librerías principales

Tecnología	Versión	Rol	Por qué se eligió
Node.js	24.16.0	Entorno de ejecución y tooling	LTS, compatible con Vite 8 y el ecosistema actual
npm	11.13.0	Gestión de dependencias	Incluido con Node; lockfile reproducible
React	19.3.1	Biblioteca de interfaz	Componentes declarativos, ecosistema maduro y buen soporte de TypeScript
React DOM	19.3.1	Renderizado web	Integración oficial de React con el DOM
Vite	8.3.4	Dev server y bundler	Arranque rápido, HMR, proxy de desarrollo y build optimizado
TypeScript	7.0.2	Lenguaje tipado	Contratos claros, refactorización segura y modo estricto
React Router DOM	7.18.4	Enrutamiento SPA	Rutas protegidas, layouts anidados y navegación declarativa
Axios	1.20.0	Cliente HTTP	Interceptores, instancia centralizada y manejo consistente de errores
React Hook Form	7.90.0	Gestión de formularios	Bajo acoplamiento, buen rendimiento y validación integrada
Zod	4.6.7	Validación de esquemas	Validación tipada y reutilizable en formularios y payloads
@hookform/resolvers	5.9.2	Puente RHF + Zod	Integración oficial entre React Hook Form y Zod
Tailwind CSS	4.3.3	Estilos	Diseño consistente sin CSS global extenso
@tailwindcss/vite	4.3.3	Integración Tailwind + Vite	Configuración mínima y compilación eficiente
jwt-decode	4.0.0	Lectura del JWT	Obtener `sub`, `role`, `user_id` y `exp` sin verificar la firma en el cliente
Vitest	5.0.5	Pruebas unitarias	Rápido, integrado con Vite y compatible con TypeScript
Testing Library	16.3.3	Pruebas de componentes	Enfoque en el comportamiento visible para el usuario
jest-dom	7.0.1	Matchers DOM	Aserciones legibles sobre el DOM
user-event	14.6.8	Simulación de interacción	Eventos más cercanos al comportamiento real del usuario
MSW	3.0.3	Mock de API en pruebas	Intercepta peticiones HTTP sin modificar los servicios
jsdom	30.1.3	Entorno DOM para pruebas	Ejecuta componentes React en Node durante Vitest
oxlint	1.89.0	Lint	Verificación rápida de reglas de React y TypeScript

3.3 Herramientas de calidad y build

- **TypeScript estricto:** `strict`, `noUnusedLocals`, `noUnusedParameters`, `noFallthroughCasesInSwitch`.
- **Vite:** build de producción con división de código por rutas (`React.lazy`).
- **Vitest + Testing Library + MSW:** pruebas de esquemas, servicios, guards y pantalla de login.
- **oxlint:** reglas de hooks y exportación de componentes.
- **Variables de entorno:** `VITE_API_BASE_URL`; `.env` fuera del control de versiones.

4. Arquitectura del frontend

4.1 Capas

Capa	Carpeta	Responsabilidad
Configuración	`src/config`	URL base, endpoints centralizados y claves de sesión
API	`src/api`	Cliente Axios, interceptores, normalización de errores y servicios por recurso
Tipos	`src/types`	Interfaces 1:1 con los DTOs del backend
Esquemas	`src/schemas`	Validaciones Zod equivalentes a las reglas del backend
Autenticación	`src/auth`	Contexto de sesión, decodificación del JWT, guards y almacenamiento del token
Hooks	`src/hooks`	Lógica reutilizable de carga de datos
Componentes	`src/components`	Layout, UI reutilizable y feedback visual
Páginas	`src/pages`	Vistas por módulo y por rol
Utilidades	`src/utils`	Formato de fechas, moneda, etiquetas, permisos y clases CSS
Pruebas	`src/tests`	Pruebas unitarias, de servicios y de componentes

4.2 Estructura de carpetas

```
frontend/
  docs/
    ANALISIS_BACKEND.md
    EVALUACION_SEGURIDAD.md
    DOCUMENTACION_FRONTEND.md
    DOCUMENTACION_FRONTEND.pdf
  scripts/
    dev.ps1
  src/
    api/
      client.ts
      errors.ts
      services/
        auth.service.ts
        materials.service.ts
        orders.service.ts
        ai.service.ts
        audit.service.ts
    auth/
      AuthContext.ts
      AuthProvider.tsx
      guards.tsx
      jwt.ts
      tokenStorage.ts
      useAuth.ts
    components/
      feedback/
      forms/ (reservada para formularios reutilizables)
      layout/
      ui/
    config/
      endpoints.ts
      env.ts
    hooks/
      useApiData.ts
    pages/
      ai/
      audit/
      auth/
      dashboard/
      errors/
      materials/
      orders/
      session/
    schemas/
```

```
      ai.schema.ts
      auth.schema.ts
      material.schema.ts
      order.schema.ts
    tests/
    types/
      api.ts
    utils/
      cn.ts
      format.ts
      labels.ts
      permissions.ts
  .env
  .env.example
  .env.test
  .gitignore
  index.html
  package.json
  tsconfig.json
  vite.config.ts
  vitest.config.ts
```

4.3 Flujo de una operación

```
Usuario
  → Componente de página
  → Hook / estado local
  → Servicio API (src/api/services)
  → Axios (cliente centralizado)
  → Proxy de Vite (/api) o URL absoluta
  → Backend FastAPI
  → Respuesta JSON
  → Normalización de errores (si aplica)
  → Estado de la página
  → Interfaz actualizada
```

Los componentes visuales nunca llaman a `axios` directamente. Toda petición pasa por un servicio, lo que facilita pruebas, cambios de contrato y manejo uniforme de errores.

4.4 Rendimiento y carga por rutas

Las páginas se cargan con `React.lazy` y `Suspense`. El build genera fragmentos separados para login, dashboard, materiales, pedidos, IA, auditoría y sesión. Esto reduce el paquete inicial y mejora el tiempo de carga. El layout permanece visible mientras se carga el módulo solicitado.

4.5 Diseño responsive

- Barra lateral fija en escritorio y menú desplegable en móvil.
- Tablas con desplazamiento horizontal en pantallas pequeñas.
- Formularios en una o dos columnas según el ancho.
- Mensajes, botones y acciones adaptados a teclado y lectores de pantalla.

5. Autenticación y sesión

5.1 Flujo de login

1. El usuario escribe `username` y `password`.
2. El formulario valida que ambos campos no estén vacíos y que el usuario no supere 50 caracteres.
3. Se envía `POST /api/v1/auth/token` con `Content-Type: application/json`.
4. El backend responde `200` con `access_token` y `token_type`.
5. El frontend decodifica el JWT para obtener `sub`, `role`, `user_id` y `exp`.
6. Se guarda el token en `sessionStorage` y se actualiza el contexto de sesión.
7. El usuario es redirigido a `/` o a la ruta que intentaba visitar.

Petición real

```
POST /api/v1/auth/token
Content-Type: application/json

{
  "username": "admin",
  "password": "AdminPass123!"
}
```

Respuesta real

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... ",
  "token_type": "bearer"
}
```

Claims del JWT

Claim	Tipo	Uso en el frontend
`sub`	string	Nombre de usuario mostrado en la interfaz
`role`	string	Control de menú, rutas y acciones
`user_id`	number	Identificador de usuario en "Mi sesión"
`exp`	number	Detección de expiración (60 minutos)

El frontend no valida la firma del token; solo lee sus datos para la interfaz. La autorización real siempre la realiza el backend.

5.2 Almacenamiento del token

Se eligió `sessionStorage` porque:

- El backend solo entrega el JWT en el cuerpo JSON; no ofrece cookies `HttpOnly`.
- `sessionStorage` está aislado por pestaña y se borra al cerrarla.

- Es menos persistente que `localStorage`.
- Permite sobrevivir a una recarga de página dentro de la misma pestaña.

Riesgo residual: cualquier token accesible por JavaScript es vulnerable ante XSS. El frontend mitiga el riesgo evitando `dangerouslySetInnerHTML` y no insertando HTML de la API. La solución definitiva requiere cookies `HttpOnly`, `Secure` y `SameSite`, junto con protección CSRF, lo que exige cambios en el backend.

5.3 Expiración y cierre de sesión

- Al iniciar la aplicación se lee el token almacenado.
- Si el token no se puede decodificar o `exp` ya pasó, se elimina.
- El interceptor de respuestas detecta `401` y emite un evento de sesión expirada.
- El contexto limpia token y usuario; los guards redirigen a `/login`.
- El botón "Cerrar sesión" elimina el token local y navega al login.
- No existe logout del servidor ni revocación de tokens.

5.4 Registro

El formulario solicita:

- `username` (3–50 caracteres).
- `email` con formato válido.
- `password` (mínimo 8 caracteres).
- Confirmación de contraseña.

El frontend **no envía el campo** `role`. El backend aplica su valor por defecto `cliente`. Esto evita exponer en la interfaz la escalada de privilegios que permite el endpoint actual.

Petición real

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "username": "nuevo_cliente",
  "email": "nuevo@kohem.com",
  "password": "Password123!"
}
```

Respuesta real

```
{
  "id": 3,
  "username": "nuevo_cliente",
  "email": "nuevo@kohem.com",
  "role": "cliente",
  "is_active": true
}
```

5.5 Guards de ruta

Guard	Comportamiento
`PublicOnlyRoute`	Si ya hay sesión, redirige al panel; si no, permite login/registro
`ProtectedRoute`	Sin sesión redirige a `/login`; con rol no autorizado redirige a `/forbidden`

Rutas protegidas por rol:

- `/materials/new`: solo `admin`.
- `/materials/:id/edit`: `admin` y `analista`.
- `/orders/new`: solo `cliente`.
- `/audit-logs`: `admin` y `analista`.

6. Conexión con el backend

6.1 URL base y variables de entorno

El frontend usa una única variable:

```
VITE_API_BASE_URL=/api/v1
```

En desarrollo, Vite reenvía las peticiones que comienzan con `/api` a `http://127.0.0.1:8000`:

```
server: {
  port: 5173,
  proxy: {
    '/api': {
      target: 'http://127.0.0.1:8000',
      changeOrigin: true,
    },
  },
}
```

El proxy también se configuró para `npm run preview`, de modo que la vista previa del build funcione en local sin CORS.

En producción, si el frontend se sirve desde otro origen, debe configurarse CORS en FastAPI. El proxy de Vite **no sustituye** esa configuración.

6.2 Cliente Axios

El cliente centralizado:

- Usa `baseURL` desde `VITE_API_BASE_URL`.
- Usa el adaptador `fetch` de Axios.
- Agrega `Authorization: Bearer <token>` si hay token almacenado.
- Detecta `401` en respuestas que no sean el login.
- Limpia el token y emite el evento `kohem:unauthorized`.
- No registra tokens ni datos sensibles en consola.

6.3 Interceptores y manejo de sesión

```
Solicitud → Request interceptor → Agrega Bearer → Backend
Respuesta ← Response interceptor ← 401 → Limpia token + evento
```

El login se excluye del cierre automático para poder mostrar “Usuario o contraseña incorrectos” en lugar de “Sesión expirada”.

6.4 Normalización de errores

HTTP	Mensaje mostrado	Comportamiento
400	Detalle de negocio del backend	Se muestra el texto seguro de stock, material o duplicado
401	“Tu sesión expiró. Inicia sesión nuevamente.”	Limpia sesión y redirige al login
403	“No tienes permisos para realizar esta acción.”	Se muestra en la vista o formulario
404	“El recurso solicitado no existe.”	Estado de error con reintento
409	Detalle de negocio del backend	Conflicto de estado de pedido
415	Mensaje genérico de operación	No se usa en el flujo normal
422	“Revisa los campos marcados…”	Se mapean errores por campo
429	“Demasiados intentos…”	Contemplado aunque el backend no lo emite
500	“Ocurrió un error interno…”	Mensaje genérico
502	“El servicio no está disponible. Verifica que el backend esté en ejecución…”	Backend caído o proxy sin destino
503/504	Servicio no disponible / tiempo agotado	Mensajes específicos
Red	“No se pudo conectar con el servidor…”	Revisión de conexión

El backend **no devuelve** `request_id`, por lo que no existe un código de soporte que mostrar al usuario.

7. Servicios API y payloads

7.1 Endpoints consumidos

Método	Endpoint	Servicio	Roles
POST	`/api/v1/auth/token`	`auth.service.ts`	Público
POST	`/api/v1/auth/register`	`auth.service.ts`	Público
GET	`/api/v1/materials`	`materials.service.ts`	Público
GET	`/api/v1/materials/{id}`	`materials.service.ts`	Público
POST	`/api/v1/materials`	`materials.service.ts`	`admin`
PUT	`/api/v1/materials/{id}`	`materials.service.ts`	`admin`, `analista`
DELETE	`/api/v1/materials/{id}`	`materials.service.ts`	`admin`
POST	`/api/v1/orders`	`orders.service.ts`	`cliente`
GET	`/api/v1/orders`	`orders.service.ts`	Autenticado
GET	`/api/v1/orders/{id}`	`orders.service.ts`	Propietario o staff
PATCH	`/api/v1/orders/{id}/status`	`orders.service.ts`	Autenticado según rol
POST	`/api/v1/ai/chat`	`ai.service.ts`	Autenticado
GET	`/api/v1/audit-logs`	`audit.service.ts`	`admin`, `analista`

7.2 Materias primas

Listar

```
GET /api/v1/materials?skip=0&limit=100
```

Respuesta:


```
[
  {
    "id": 1,
    "name": "Aceite esencial de menta",
    "category": "Aceites esenciales",
    "unit": "kg",
    "price": 85000.0,
    "stock": 50.0,
    "description": "Materia prima para formulaciones cosméticas.",
    "is_active": true
  }
]
```

Crear

```
POST /api/v1/materials
Authorization: Bearer <token-admin>
Content-Type: application/json

{
  "name": "Glicerina vegetal",
  "category": "Humectantes",
  "unit": "kg",
  "price": 18000,
  "stock": 100,
  "description": "Materia prima para formulaciones cosméticas."
}
```

Validaciones del formulario y del backend:

Campo	Regla
`name`	2–120 caracteres
`category`	2–80 caracteres
`unit`	1–20 caracteres
`price`	Mayor que 0
`stock`	Mayor o igual a 0
`description`	Máximo 500 caracteres; opcional

Editar

```
PUT /api/v1/materials/2
Authorization: Bearer <token-admin-o-analista>
Content-Type: application/json

{
  "name": "Glicerina vegetal refinada",
  "category": "Humectantes",
  "unit": "kg",
  "price": 19500,
  "stock": 120,
  "description": "Materia prima actualizada."
}
```

El frontend envía el objeto completo para evitar ambigüedad, aunque el DTO del backend permite campos opcionales.

Eliminar

```
DELETE /api/v1/materials/2
Authorization: Bearer <token-admin>
```

Respuesta: `204 No Content`. Es un borrado físico; no existe desactivación lógica en el backend. La interfaz muestra una confirmación explícita antes de ejecutarlo.

7.3 Pedidos

Crear pedido

```
POST /api/v1/orders
Authorization: Bearer <token-cliente>
Content-Type: application/json

{
  "items": [
    { "raw_material_id": 1, "quantity": 5 },
    { "raw_material_id": 2, "quantity": 10 }
  ],
  "notes": "Pedido para formulación cosmética."
}
```

Respuesta:

```
{
  "id": 1,
  "customer_id": 2,
  "status": "pendiente",
  "total": 605000.0,
  "notes": "Pedido para formulación cosmética.",
  "items": [
    {
      "id": 1,
      "raw_material_id": 1,
      "quantity": 5.0,
      "unit_price": 85000.0,
      "subtotal": 425000.0
    },
    {
      "id": 2,
      "raw_material_id": 2,
      "quantity": 10.0,
      "unit_price": 18000.0,
      "subtotal": 180000.0
    }
  ],
  "created_at": "2026-10-20T10:00:00",
  "updated_at": "2026-10-20T10:00:00"
}
```


Reglas internas del frontend antes de enviar:

- Debe existir al menos una línea.
- `raw_material_id` debe ser un entero positivo.
- `quantity` debe ser mayor que cero.
- No se permiten líneas duplicadas del mismo material.
- La cantidad agregada no puede superar el stock mostrado.
- El material debe estar activo.
- Las notas tienen un máximo de 500 caracteres.

El total definitivo siempre se toma de la respuesta del servidor. El “total estimado” del carrito es solo una referencia visual y no se usa como dato de confianza.

Listar pedidos

```
GET /api/v1/orders?skip=0&limit=100
Authorization: Bearer <token>
```

- `cliente`: el backend devuelve solo sus pedidos.
- `admin` y `analista`: devuelve todos.
- La lista se ordena por id descendente en el backend.

Consultar pedido

```
GET /api/v1/orders/1
Authorization: Bearer <token>
```

Un cliente que consulta un pedido ajeno recibe `403`.

Cambiar estado

```
PATCH /api/v1/orders/1/status
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "confirmado"
}
```

Estados válidos: `pendiente`, `confirmado`, `preparando`, `enviado`, `entregado`, `cancelado`.

- `cliente`: solo puede enviar `cancelado` y únicamente si el pedido no está en estado final.
- `admin` y `analista`: pueden enviar cualquier estado si el pedido no está `entregado` ni `cancelado`.
- Al cancelar, el backend devuelve el stock reservado.
- El backend no implementa una matriz de transición; la interfaz respeta esa realidad y no restringe estados que el servidor aceptaría.

7.4 Asesoría IA

```
POST /api/v1/ai/chat
Authorization: Bearer <token>
Content-Type: application/json

{
  "message": "Necesito asesoría para formular un producto cosmético."
}
```

Respuesta:

```
{
  "response": "Puedo registrar tu consulta, pero este caso requiere revisión de un asesor de Kohem Chemicals.",
  "transfer_to_employee": true,
  "reason": "La consulta puede requerir conocimiento técnico o intervención humana."
}
```

El agente es simulado y funciona por palabras clave. No hay integración con un modelo de lenguaje externo ni persistencia de la conversación.

7.5 Auditoría

```
GET /api/v1/audit-logs?limit=100
Authorization: Bearer <token-admin-o-analista>
```

Respuesta:

```
[
  {
    "id": 1,
    "user_id": 2,
    "action": "CREATE",
    "entity": "order",
    "entity_id": 1,
    "detail": "Pedido creado por cliente 2; total=605000.0",
    "created_at": "2026-10-20T10:00:00"
  }
]
```

La vista es de solo lectura. No existen filtros ni paginación con total; la API devuelve hasta 100 registros.

8. Funcionalidades y pantallas

8.1 Login

Ruta: `/login`

Elementos:

- Campo Usuario.
- Campo Contraseña.
- Botón Entrar.
- Enlace a registro.
- Credenciales de demostración visibles solo en desarrollo.

Comportamiento:

- Valida campos obligatorios.
- Envía JSON al endpoint de token.
- Guarda el JWT y redirige.
- Con `401`, muestra "Usuario o contraseña incorrectos".
- Con backend caído (`502`), muestra que el servicio no está disponible.

8.2 Registro

Ruta: `/register`

Elementos:

- Usuario.
- Correo electrónico.
- Contraseña.
- Confirmación de contraseña.
- Enlace a login.

Comportamiento:

- Valida longitudes, correo y coincidencia de contraseñas.
- No muestra selector de rol.
- Crea la cuenta como `cliente`.
- Redirige al login con mensaje de éxito.

8.3 Layout y menú

Elementos:

- Barra lateral con logo y nombre.
- Navegación: Panel, Materiales, Pedidos, Asesoría IA, Auditoría y Mi sesión.
- Tarjeta de usuario con rol.
- Botón Cerrar sesión.
- Menú móvil desplegable.

El enlace de Auditoría solo aparece para `admin` y `analista`.

8.4 Dashboard

Ruta: `/`

Elementos:

- Saludo con nombre de usuario y rol.
- Acciones rápidas: Nuevo pedido (cliente) o Nuevo material (admin).
- Tarjetas: materiales, pedidos, pedidos en curso, pedidos entregados.
- Tabla de pedidos recientes.
- Accesos rápidos a materiales, pedidos, IA y auditoría.
- Contador de eventos de auditoría para staff.

Comportamiento interno:

- Consulta materiales y pedidos en paralelo.
- Consulta auditoría solo si el rol lo permite.
- Calcula métricas sobre los hasta 100 registros devueltos.
- No usa agregados del servidor porque no existen.

8.5 Materias primas

Ruta: `/materials`

Opciones:

- Buscar por nombre o categoría en el cliente.
- Ver nombre, descripción, categoría, precio, unidad, stock y estado.
- Crear material (solo `admin`).
- Editar material (`admin` y `analista`).
- Eliminar material (solo `admin`) con confirmación.
- "Pedir" para clientes, deshabilitado si el material está inactivo o sin stock.

Peticiones internas:

- `GET /materials?limit=100` al cargar.
- `POST /materials` al crear.
- `PUT /materials/{id}` al editar.
- `DELETE /materials/{id}` al eliminar.

La eliminación es física y se advierte en el diálogo de confirmación.

8.6 Formulario de material

Rutas: `/materials/new` y `/materials/:id/edit`

Campos:

- Nombre.
- Categoría.

- Unidad de medida.
- Precio.
- Stock.
- Descripción.

Comportamiento:

- En edición carga el material y rellena el formulario.
- Valida con Zod antes de enviar.
- Mapea errores `422` del servidor a los campos.
- Al guardar, vuelve al listado con mensaje de éxito.

8.7 Pedidos

Ruta: `/orders`

Opciones:

- Buscar por id de pedido; staff también por id de cliente.
- Filtrar por estado.
- Ver id, cliente (staff), estado, número de líneas, total y fecha.
- Abrir detalle.
- Crear pedido (solo cliente).

Petición interna: `GET /orders?limit=100`.

8.8 Nuevo pedido

Ruta: `/orders/new` (solo cliente)

Elementos:

- Selector de materia prima activa.
- Campo de cantidad.
- Botón Agregar.
- Tabla del carrito con cantidades editables y opción Quitar.
- Campo de notas.
- Resumen con total estimado.
- Botón Registrar pedido.

Comportamiento:

- Impide duplicar un material.
- Valida stock y cantidad antes de agregar y antes de enviar.
- Muestra el stock disponible y la unidad.
- Envía `POST /orders` con `items` y `notes`.
- Al crear, navega al detalle y muestra el total devuelto por el servidor.
- Si el servidor rechaza por stock, muestra el detalle de negocio.

8.9 Detalle de pedido

Ruta: `/orders/:id`

Elementos:

- Estado, fechas y total.
- Tabla de materiales con nombre, cantidad, precio unitario y subtotal.
- Notas del pedido.
- Cliente ID para staff.
- Acciones según rol.

Cliente:

- Puede cancelar si el pedido no está entregado ni cancelado.
- La cancelación pide confirmación y advierte que se devuelve stock.

Staff:

- Puede seleccionar un nuevo estado y actualizar.
- La cancelación también pide confirmación.
- Si el pedido está en estado final, no se muestran acciones.

Peticiones internas:

- `GET /orders/{id}`.
- `GET /materials?limit=100` para resolver nombres.
- `PATCH /orders/{id}/status`.

8.10 Asesoría IA

Ruta: `/ai`

Elementos:

- Historial de mensajes en la sesión actual.
- Sugerencias rápidas.
- Campo de consulta con límite de 1000 caracteres.
- Botón Enviar.
- Indicador de respuesta en curso.
- Insignia “Transferencia a empleado” cuando corresponde.

No hay historial persistente ni conexión con un empleado real; solo se muestra la marca que devuelve el backend.

8.11 Auditoría

Ruta: `/audit-logs` (solo `admin` y `analista`)

Elementos:

- Tabla de logs con id, fecha, usuario, acción, entidad y detalle.
- Estado vacío si no hay eventos.
- Sin acciones de edición.

Petición interna: `GET /audit-logs?limit=100`.

8.12 Mi sesión

Ruta: `/session`

Elementos:

- Usuario.
- Rol.
- ID de usuario.
- Fecha de expiración del token.
- Botón Cerrar sesión.

No existe edición de perfil ni endpoint `/users/me` en el backend.

8.13 Errores 403 y 404

- `/forbidden`: se muestra cuando un rol intenta acceder a una ruta no permitida.
- Cualquier ruta desconocida muestra la página 404 con enlace al panel.

9. Roles y permisos

Operación	admin	analista	cliente
Iniciar sesión	Sí	Sí	Sí
Registrarse	Sí	Sí	Sí
Ver materiales	Sí	Sí	Sí
Crear material	Sí	No	No
Editar material	Sí	Sí	No
Eliminar material	Sí	No	No
Crear pedido	No	No	Sí
Ver todos los pedidos	Sí	Sí	No
Ver pedidos propios	—	—	Sí
Ver pedido ajeno	Sí	Sí	No
Cambiar estado a cualquier valor	Sí	Sí	No
Cancelar pedido	Sí	Sí	Sí (propio)
Usar agente IA	Sí	Sí	Sí
Ver auditoría	Sí	Sí	No

La interfaz oculta o deshabilita acciones no permitidas, pero el backend vuelve a validar cada solicitud.

10. Reglas de negocio reflejadas en la interfaz

1. **Stock al crear:** el stock se valida y descuenta al registrar el pedido, no al confirmarlo.
2. **Cancelación:** devuelve el stock reservado.
3. **Estados finales:** `entregado` y `cancelado` no admiten cambios.
4. **Cliente:** solo puede cancelar sus propios pedidos y solo si no están en estado final.
5. **Staff:** puede cambiar a cualquier estado mientras el pedido no sea final.
6. **Total:** se muestra el total del servidor; el estimado del carrito es referencial.
7. **Materiales inactivos:** no se ofrecen para nuevos pedidos.
8. **Duplicados:** el frontend impide dos líneas del mismo material para evitar el riesgo de sobreventa del backend.
9. **Borrado:** la eliminación de materiales es física; se advierte al usuario.
10. **Auditoría:** solo registra creación y cambio de estado de pedidos.
11. **Citas y geografía:** no existen en el backend y no se simulan.
12. **Fechas:** el backend envía fechas ISO sin zona horaria; el frontend las interpreta como UTC y las muestra en hora local.

11. Manejo de errores y estados de interfaz

11.1 Estados

Estado	Implementación
Carga	`LoadingScreen`, `Spinner` y botones con `isLoading`
Vacío	`EmptyState` en listas, carrito, auditoría y pedidos
Error	`ErrorState` con mensaje y botón Reintentar
Éxito	`Alert` con mensaje posterior a crear, editar, cancelar o cambiar estado
Confirmación	`ConfirmDialog` para eliminar materiales y cancelar pedidos

11.2 Errores por campo

Cuando el backend devuelve `422`, el frontend recorre `detail[].loc`, obtiene el nombre del campo y asigna un mensaje genérico "Valor inválido según el servidor". No muestra el mensaje técnico de Pydantic, que puede estar en inglés o revelar detalles de implementación.

11.3 Errores de negocio

Los errores `400` y `409` contienen mensajes de negocio en español, por ejemplo:

- "Stock insuficiente para 'Glicerina vegetal'. Disponible: 100.0, solicitado: 150.0."
- "El pedido ya está en un estado final y no puede modificarse."

Se muestran como texto escapado por React, sin interpretar HTML.

12. Seguridad del frontend

Control	Implementación	Riesgo residual
Almacenamiento del token	`sessionStorage` por pestaña	XSS podría leer el token; se recomienda cookie HttpOnly
Exposición de secretos	Solo `VITE_API_BASE_URL`; `.env` ignorado	Las variables `VITE_*` son públicas por diseño
XSS	Sin `dangerouslySetInnerHTML`; React escapa texto	Falta CSP en el despliegue
Autorización	Guards y permisos por rol	El backend sigue siendo la autoridad
Expiración	Validación de `exp` y cierre en `401`	No hay revocación del servidor
Logs	Sin tokens ni datos sensibles en consola	—
CORS	Proxy de Vite en desarrollo	Producción requiere CORS en FastAPI
CSRF	No se usan cookies de sesión	Si se migra a cookies, añadir CSRF
Contraseñas	Mínimo 8 caracteres validado en cliente y servidor	El backend no exige complejidad

Hallazgos del backend que afectan al frontend

1. **Crítico:** el registro público acepta `role`, incluido `admin`. Verificado en prueba de humo.
2. **Alto:** no hay CORS configurado.
3. **Alto:** no hay rate-limit ni lockout en login.
4. **Alto:** `SECRET_KEY` tiene valor por defecto si no existe `.env`.
5. **Medio:** no hay refresh ni revocación de tokens.
6. **Medio:** borrado físico de materiales.
7. **Medio:** riesgo de sobreventa por concurrencia y por líneas duplicadas.
8. **Medio:** ausencia de `request_id`.
9. **Bajo:** Swagger y OpenAPI públicos.
10. **Bajo:** política de contraseñas débil.

13. Pruebas y verificación

13.1 Pruebas automatizadas

Archivo	Cobertura
`schemas.test.ts`	Validaciones de login, registro, materiales, pedidos e IA
`permissions.test.ts`	Matriz de permisos por rol
`errors.test.ts`	Normalización de 422, 409, 403, 502, red y errores no Axios
`jwt.test.ts`	Decodificación de claims, rol inválido, token malformado y expiración
`api-services.test.ts`	Login JSON, registro sin `role`, listado de materiales, Bearer y PATCH de estado
`guards.test.tsx`	Redirección sin sesión, acceso permitido y acceso denegado por rol
`login-page.test.tsx`	Login exitoso y mensaje claro con credenciales inválidas

Resultado:

```
Test Files  7 passed (7)
Tests       36 passed (36)
```

13.2 Comandos ejecutados

```
npm install       # instalación de dependencias
npm run lint      # oxlint, sin advertencias
npm run build     # typecheck + build de producción
npm run test      # 36 pruebas en verde
```

13.3 Pruebas de humo contra el backend real

Se levantó el backend con una base SQLite temporal para no alterar la base original:

- Login de `admin` y `cliente` → `200`.
- Creación de un `analista` vía API → `201`.
- `analista` edita material → `200`.
- `analista` crea/elimina material → `403`.
- `analista` consulta pedidos y auditoría → `200`.
- Pedido del cliente: total `18000`, stock `100 → 99`.
- Cancelación: stock restaurado a `100`.
- Cliente crea material → `403`.
- Cliente consulta auditoría → `403`.
- Admin consulta auditoría → `200`.
- CORS: `Access-Control-Allow-Origin` vacío.
- OpenAPI: 10 rutas reales.
- Vite sirvió la aplicación y el proxy devolvió los materiales.

- La base original `kohem.db` quedó intacta.

14. Ejecución y uso

14.1 Requisitos

- Node.js 24 LTS o superior.
- npm 11 o superior.
- Backend FastAPI ejecutable desde la carpeta `MVP`.
- Python y dependencias del backend para el entorno virtual.

14.2 Instalación

```
cd "MVP/frontend"
npm install
copy .env.example .env
```

14.3 Arranque conjunto en Windows

```
npm run dev:all
```

El script:

1. Comprueba si el backend responde en `http://127.0.0.1:8000`.
2. Si no responde, inicia `uvicorn` desde `../venv`.
3. Ejecuta `npm run dev`.
4. Al cerrar Vite, detiene el backend que haya iniciado.

14.4 Arranque separado

Terminal 1:

```
cd "MVP"
venv\Scripts\python -m uvicorn app.presentation.main:app --reload
```

Terminal 2:

```
cd "MVP/frontend"
npm run dev
```

Abrir `http://localhost:5173`.

14.5 Scripts disponibles

Script	Función
`npm run dev`	Servidor de desarrollo con proxy
`npm run dev:all`	Backend + frontend en Windows
`npm run build`	Typecheck y build de producción
`npm run preview`	Vista previa del build con proxy
`npm run lint`	Análisis estático con oxlint
`npm run typecheck`	Verificación de tipos
`npm run test`	Pruebas automatizadas
`npm run test:watch`	Pruebas en modo watch
`npm run test:coverage`	Cobertura de pruebas

14.6 Usuarios de prueba

Rol	Usuario	Contraseña
Administrador	`admin`	`AdminPass123!`
Cliente	`cliente`	`ClientPass123!`
Analista	No seedeado	Crear vía API con `"role": "analista"`

El formulario del frontend no permite elegir rol. Para probar el rol `analista` se puede usar Swagger o una petición directa al backend.

14.7 Solución de problemas

Síntoma	Causa probable	Solución
"El servicio no está disponible..."	Backend caído o proxy sin destino	Iniciar backend o usar `npm run dev:all`
"No se pudo conectar con el servidor..."	URL de API incorrecta o red	Revisar `VITE_API_BASE_URL` y conexión
`401` al navegar	Token expirado	Volver a iniciar sesión
`403`	Rol sin permiso	Usar la cuenta correcta
`502` en login	Vite no encuentra el backend en `8000`	Verificar `http://127.0.0.1:8000/`
Error de CORS en producción	Backend sin `CORSMiddleware`	Configurar CORS en FastAPI

15. Limitaciones y trabajo futuro

15.1 Limitaciones actuales

- No hay gestión de usuarios ni edición de perfil.
- No hay citas ni agendamiento.
- No hay selectores geográficos.
- No hay refresh token ni revocación.
- No hay rate-limit ni bloqueo de login.
- No hay borrado lógico de materiales.
- No hay filtros ni paginación con total en el backend.
- No se muestra el nombre del cliente en pedidos, solo `customer_id`.
- La auditoría solo cubre pedidos.
- El dashboard no usa agregados del servidor.

15.2 Cambios recomendados en el backend

1. Restringir `role` en el registro público.
2. Añadir `CORSMiddleware` con el origen exacto del frontend.
3. Configurar una `SECRET_KEY` obligatoria y fuerte.
4. Implementar rate-limit y lockout en login.
5. Añadir refresh token con rotación y revocación.
6. Implementar desactivación lógica de materiales.
7. Añadir `request_id` a respuestas y errores.
8. Agregar transacciones o bloqueos para evitar sobreventa.
9. Implementar endpoints de usuarios, citas y geografía si el alcance lo requiere.
10. Añadir auditoría de materiales, usuarios y accesos denegados.

15.3 Mejoras futuras del frontend

- Migrar la sesión a cookies `HttpOnly` cuando el backend lo soporte.
- Añadir CSP y cabeceras de seguridad en el despliegue.
- Incorporar pruebas E2E con Playwright para los flujos por rol.
- Añadir filtros y paginación cuando el backend los exponga.
- Implementar un centro de notificaciones para transferencias de la IA.
- Internacionalización formal si se requiere más de un idioma.

16. Conclusiones

El frontend desarrollado cumple el objetivo de entregar un MVP funcional y seguro dentro de las capacidades reales del backend. La arquitectura por capas, el tipado estricto y la validación con Zod reducen errores de integración; los guards y la matriz de permisos ordenan la experiencia por rol; y el manejo centralizado de errores evita exponer detalles técnicos al usuario.

La aplicación fue verificada con compilación, lint, 36 pruebas automatizadas y pruebas de humo contra el backend real. El sistema queda listo para demostración y revisión, con una lista clara de limitaciones y de cambios necesarios para un despliegue de producción alineado con controles de seguridad más estrictos.

17. Anexos

Anexo A. Endpoints reales del backend

Método	Ruta	Auth	Roles
GET	`/`	No	Público
POST	`/api/v1/auth/register`	No	Público
POST	`/api/v1/auth/token`	No	Público
GET	`/api/v1/materials`	No	Público
GET	`/api/v1/materials/{id}`	No	Público
POST	`/api/v1/materials`	Sí	`admin`
PUT	`/api/v1/materials/{id}`	Sí	`admin`, `analista`
DELETE	`/api/v1/materials/{id}`	Sí	`admin`
POST	`/api/v1/orders`	Sí	`cliente`
GET	`/api/v1/orders`	Sí	Autenticado
GET	`/api/v1/orders/{id}`	Sí	Propietario o staff
PATCH	`/api/v1/orders/{id}/status`	Sí	Autenticado según rol
POST	`/api/v1/ai/chat`	Sí	Autenticado
GET	`/api/v1/audit-logs`	Sí	`admin`, `analista`

Anexo B. Variables de entorno

Variable	Ejemplo	Uso
`VITE_API_BASE_URL`	`/api/v1`	URL base de la API en desarrollo con proxy
`VITE_API_BASE_URL`	`http://127.0.0.1:8000/api/v1`	URL absoluta para producción, requiere CORS

Anexo C. Comandos de verificación

```
npm install
npm run lint
npm run build
npm run test
```

Anexo D. Mapa de archivos clave

Archivo	Función
`src/api/client.ts`	Instancia Axios e interceptores
`src/api/errors.ts`	Normalización de errores HTTP
`src/auth/AuthProvider.tsx`	Estado de sesión y login/logout
`src/auth/guards.tsx`	Rutas protegidas y públicas
`src/auth/tokenStorage.ts`	Lectura/escritura en `sessionStorage`
`src/config/endpoints.ts`	Endpoints centralizados
`src/config/env.ts`	URL base y claves de sesión
`src/schemas/*.ts`	Validaciones Zod
`src/types/api.ts`	Contratos TypeScript de la API
`src/pages/*`	Vistas por módulo
`src/components/ui/*`	Componentes reutilizables
`vite.config.ts`	Proxy de desarrollo y preview
`scripts/dev.ps1`	Arranque conjunto backend + frontend
`docs/ANALISIS_BACKEND.md`	Análisis detallado de la API
`docs/EVALUACION_SEGURIDAD.md`	Evaluación de controles de seguridad

Anexo E. Glosario

- **SPA:** aplicación de una sola página que se ejecuta en el navegador.
- **JWT:** token firmado que contiene identidad y rol.
- **Access token:** token de acceso de corta duración.
- **Refresh token:** token para renovar el access token; no existe en este backend.
- **Guard:** componente que protege una ruta según la sesión o el rol.
- **Payload:** cuerpo JSON enviado o recibido por la API.
- **Proxy:** servidor de desarrollo que reenvía peticiones al backend.
- **CORS:** política del navegador para peticiones entre orígenes distintos.
- **RBAC:** control de acceso basado en roles.
- **XSS:** ejecución de código malicioso inyectado en la página.
- **Soft delete:** borrado lógico mediante un campo de estado; no implementado.
